Attention và Transformer

Mỗi vị trí chọn thông tin cần lấy

Chương 3 · Bài 3.9

Một từ cần ngữ cảnh

“Con mèo không nhảy lên bàn vì nó quá cao.”

“nó” là cái bàn; đổi “cao” thành “mệt” thì “nó” là con mèo.

Mean cố định không cho biết token nào cần nhìn token nào.

Query hỏi · key so khớp · value cung cấp nội dung: tra key/value nhưng trả tổ hợp mềm.

Mục tiêu

- Tính attention bằng tay và bằng NumPy.
- Ghi shape của Q, K, V và đầu ra.
- Ghép multi-head attention, residual, LayerNorm và feed-forward thành một khối encoder.
- Dùng padding mask, causal mask và thông tin vị trí.

01

Một attention trên hai key

Scaled dot-product

$$\sqrt{\quad}$$

Softmax theo key cho từng query.

là độ rộng query/key; mỗi hàng của gồm các điểm so khớp của một query với mọi key.

HỎI NHANH

Trong một câu, mọi key giống hệt nhau, còn các value thì khác nhau. Output của một query bất kỳ là gì?

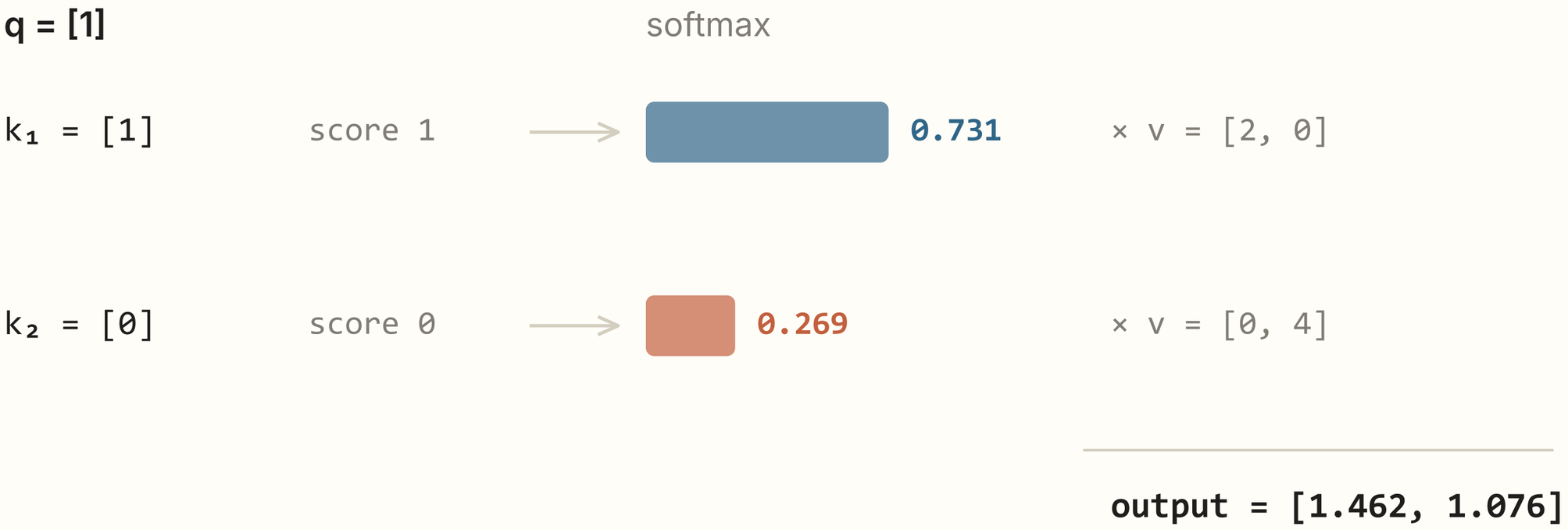
A Value đầu tiên, vì softmax phá hòa theo chỉ số

B Trung bình đều của các value

C Vector 0, vì các score triệt tiêu nhau

D Không xác định, vì softmax cần các score khác nhau

Hai score 1 và 0



Score (1, 0) qua softmax thành (0.731, 0.269), rồi trộn hai value.

Một query, hai key

Query ; key và .

, nên hệ số chia $\sqrt{}$ bằng 1. Hai score là 1 và 0.

Value và .

Output .

KẾT QUẢ

```
weights: [[0.731059, 0.268941]]
output: [[1.462117, 1.075766]]
match: True
```

`match`: trùng phép tensor của PyTorch.

HỎI NHANH

`scores` có shape `(B, Lq, Lk)`. Trong `weights = scores.softmax(dim=...)`, phải chọn `dim` nào để mỗi query phân trọng số cho các key?

A `dim=0`

B `dim=1`

C `dim=-1`

D Chiều nào cũng được, vì shape giữ nguyên

Shape của phép trộn

Tensor	Shape
	`(Lq,dk)`
	`(Lk,dk)`
	`(Lk,dv)`
Score	`(Lq,Lk)`
	`(Lq,dv)`

Trừ max trước exp để ổn định softmax.

Kiểm `weights.sum(-1)` bằng 1: shape không bắt được lỗi `dim`.

Chia score để softmax không bão hòa

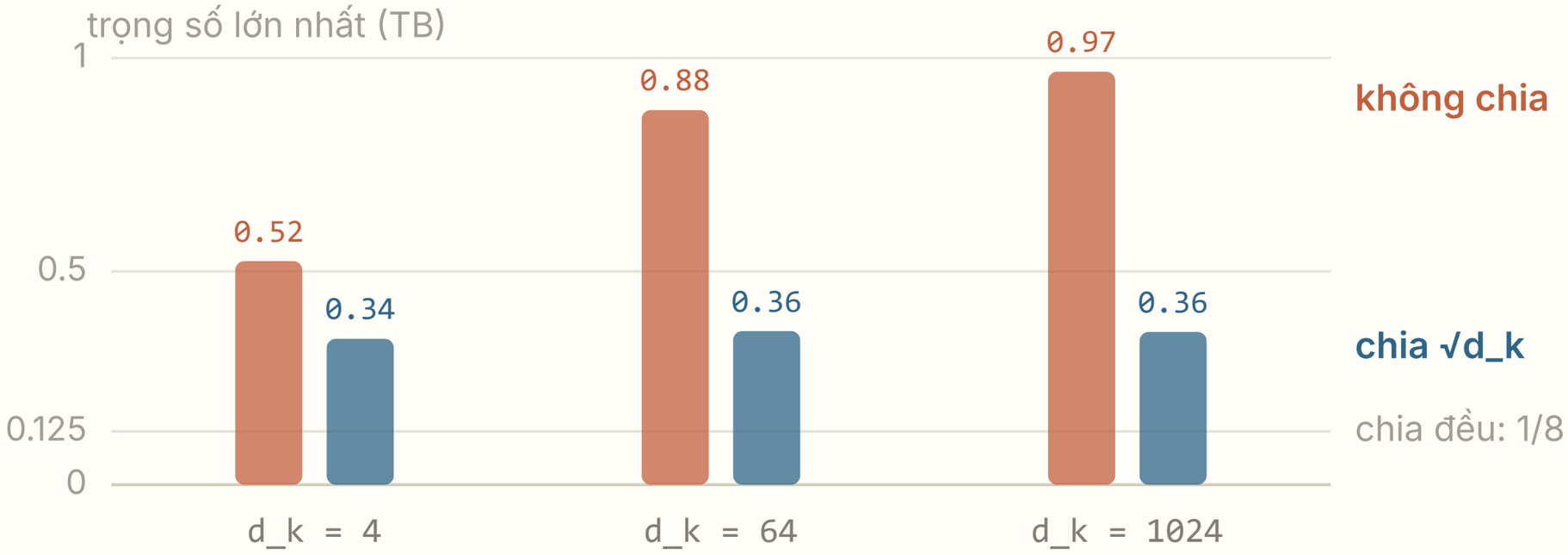
Score là tổng tích: độ lệch chuẩn cỡ $\sqrt{}$. 2000 query, mỗi query 8 key:

KẾT QUẢ

```
d_k=4     std  2.00 -> 1.00   max weight 0.524 -> 0.342
d_k=64    std  7.96 -> 0.99   max weight 0.878 -> 0.360
d_k=1024  std 31.83 -> 0.99   max weight 0.968 -> 0.358
```

Không chia: softmax gần one-hot, gradient tới key khác gần 0. Có chia: thang score không đổi theo độ rộng.

Trọng số lớn nhất theo



Không chia √d_k, một key chiếm gần hết trọng số khi d_k lớn.

HỎI NHANH

Các thành phần của và độc lập, trung bình 0, phương sai 1, với . Độ lệch chuẩn của score trước và sau khi chia cho $\sqrt{}$ là bao nhiêu?

A 256 trước, 16 sau

B 256 trước, 1 sau

C 1 trước, 1/16 sau

D 16 trước, 1 sau

02

Self-attention và mask

Cùng , ba phép chiếu

Multi-head: nhiều phép trộn, ghép rồi chiếu lại. Mỗi head một softmax: một query có phân phối cùng lúc.

tham số, không phụ thuộc ; phải chia hết cho .

HỎI NHANH

Bỏ hai phép chiếu , , dùng thẳng . Ma trận score bị ràng buộc gì?

A Đối xứng: score từ tới bằng score từ tới

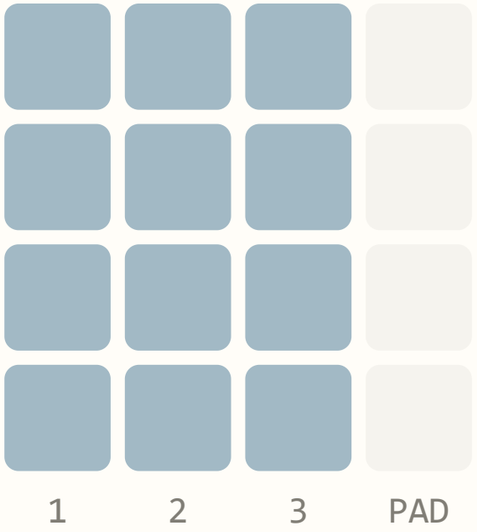
B Mọi score bằng nhau, nên attention thành mean

C Các score ngoài đường chéo luôn bằng 0

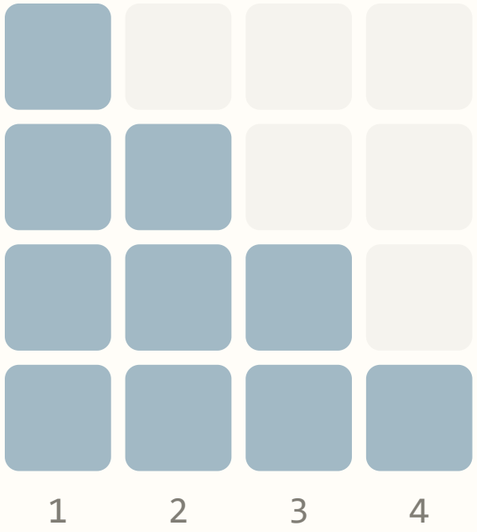
D Không bị ràng buộc gì, vì vẫn được học

Hai lý do cấm key

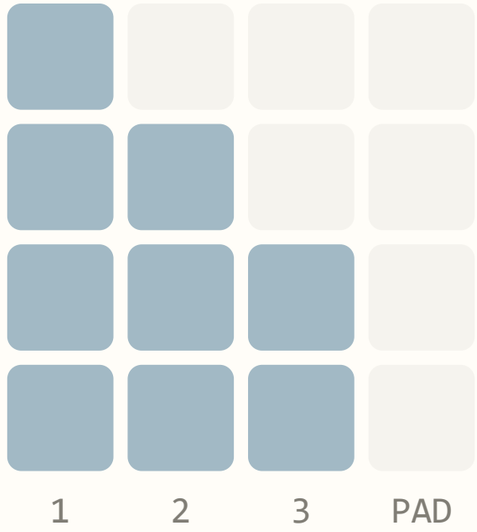
padding mask



causal mask



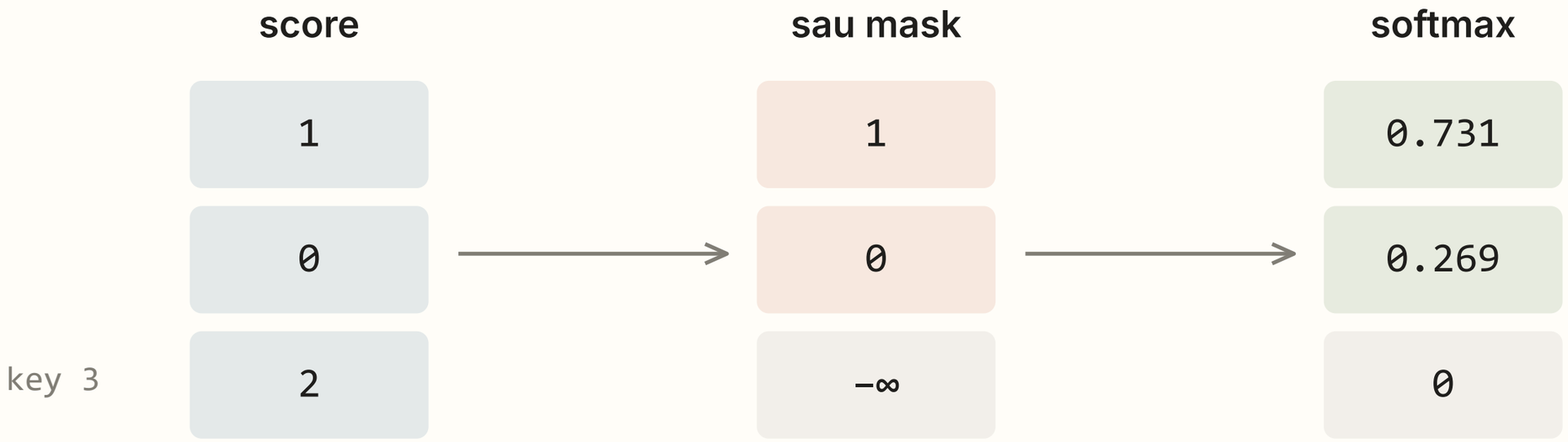
cả hai



hàng: query · cột: key · ô tô: được đọc

Padding mask bỏ cột PAD, causal mask bỏ tương lai.

Mask tác động trước softmax



cấm trước softmax: hai key còn lại tự chia lại tổng 1

Score −∞ trước softmax cho trọng số 0; tổng vẫn bằng 1.

Đọc đúng boolean API

`nn.MultiheadAttention`:

`key_padding_mask=True` là bỏ key; từ bài 3.8 truyền `~valid_mask`.

`attn_mask=True` là cấm cặp query–key.

`F.scaled_dot_product_attention`: `True` là **được** đọc.

Score vị trí cấm → trước softmax.

Không cấm mọi key của một query.

HỎI NHANH

Causal mask cho MHA được viết `torch.triu(torch.ones(L, L, dtype=torch.bool), diagonal=0)` thay vì `diagonal=1`. Chuyện gì xảy ra?

A Không đổi gì, vì đường chéo không ảnh hưởng softmax

B Mỗi query nhìn thêm được một token tương lai

C Query 0 bị cấm mọi key, nên output có thể ra NaN

D Chỉ chậm hơn một chút, vì phải cấm thêm ô

HỎI NHANH

Một hàng weights là `[0.5, 0.3, 0.2]`, trong đó key thứ ba là PAD. Code đặt weight của PAD về 0 **sau** softmax. Để khớp với cách đặt score bằng trước softmax, weight của key đầu phải bằng bao nhiêu?

A 0.5

B 0.7

C 0.8

D 0.625

03

Một encoder block

Trộn rồi biến đổi feature

`attended`: output của multi-head attention.

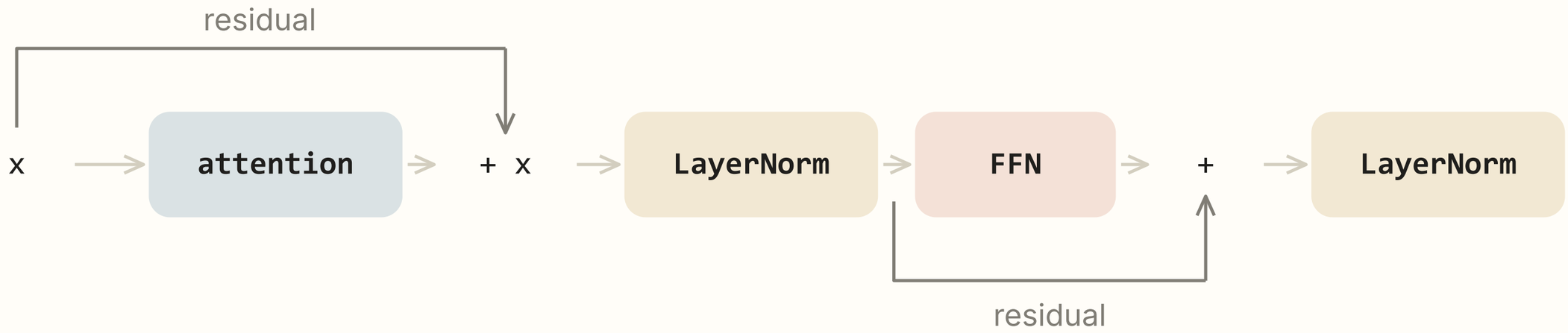
Attention → residual → norm.

FFN từng token → residual → norm.

```
x = self.norm1(x + attended)
x = self.norm2(x + self.ffn(x))
return x, weights
```

Code của bài dùng post-norm; pre-norm là thiết kế khác.

Encoder block



post-norm: norm(x + attention(x)) rồi norm(x + FFN(x)); shape (B, L, d) giữ nguyên

Encoder block: attention, cộng residual, LayerNorm; FFN, cộng residual, LayerNorm.

Shape đo được

KẾT QUẢ

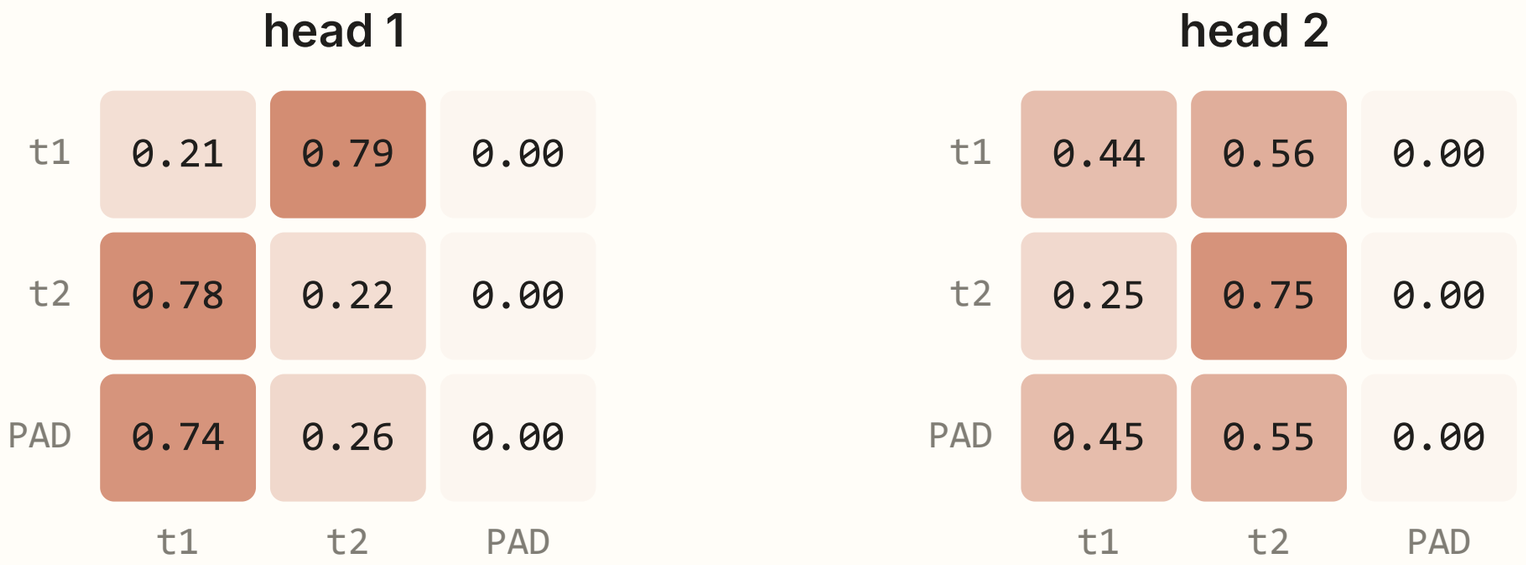
```
output: (2, 3, 4)
weights: (2, 2, 3, 3)
PAD key max weight: 0.0
weight row sums: True
```

Weights: batch, head, query, key.

PAD key bị bỏ; query PAD vẫn có thể ra vector.

Pooling/loss còn cần bỏ query PAD.

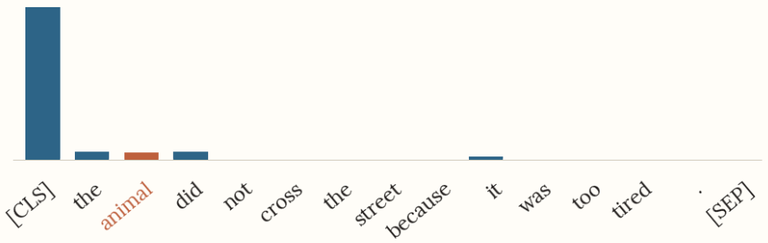
Trọng số của hai head



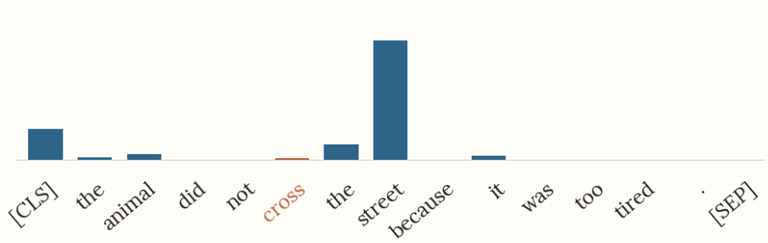
hàng: query · cột: key · cột PAD bằng 0 ở cả hai head

Hai head có phân phối riêng; cột PAD bằng 0.

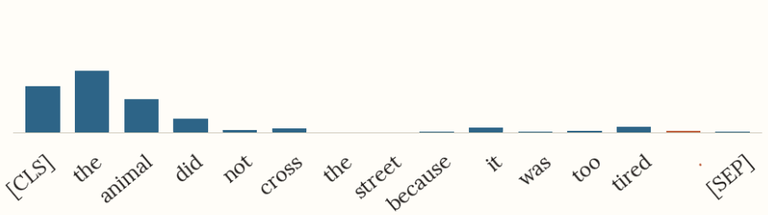
Attention của một model thật



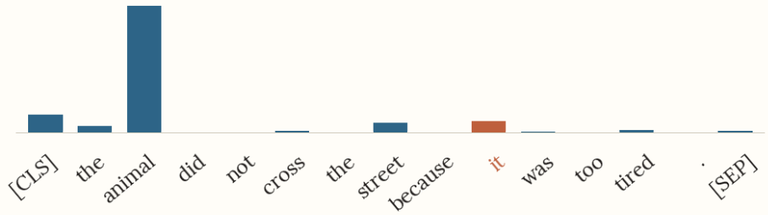
query “animal”: trọng số lớn nhất ở “[CLS]” (0.83)



query “cross”: trọng số lớn nhất ở “street” (0.65)



query “.”: trọng số lớn nhất ở “the” (0.34)



query “it”: trọng số lớn nhất ở “animal” (0.69); “animal” nhận 0.69

Một head của MiniLM: mỗi khung là một query, cột là trọng số lên từng token.

HỎI NHANH

Gọi `self.attention(x, x, x, key_padding_mask=padding_mask)` với mặc định `average_attn_weights=True`, cùng `x` có shape `(2, 3, 4)` và 2 head. `weights` có shape gì?

A `(2, 2, 3, 3)`

B `(2, 3, 3)`

C `(2, 3, 4)`

D `(3, 3)`

Test thay token tương lai

Sửa token cuối, giữ causal mask.

KẾT QUẢ

```
past unchanged: True
last position changed: True
```

Hai token đầu không dùng thông tin cuối; dòng hai là đối chứng.

HỎI NHANH

Vẫn block có causal mask ấy, nhưng lần này sửa token ở **vị trí 0** thay vì token cuối. Output ở những vị trí nào được phép thay đổi?

A Chỉ vị trí 0

B Vị trí 1 và 2, còn vị trí 0 giữ nguyên

C Không vị trí nào, vì causal mask đã chặn

D Cả ba vị trí 0, 1 và 2

04

Thông tin thứ tự

Không vị trí: hoán vị tương ứng

Eval, `dropout=0`, không mask cố định theo vị trí.

Đảo token, output đảo cùng thứ tự.

Mean cuối không phân biệt thứ tự.

KẾT QUẢ

```
equivariant without positions: True
```

Nếu có mask, phải hoán vị mask cùng token.

Giữ causal mask cố định không thỏa điều kiện này.

Thêm sin/cos theo slot

Token đảo nhưng vị trí 0, 1, 2 giữ tại slot.

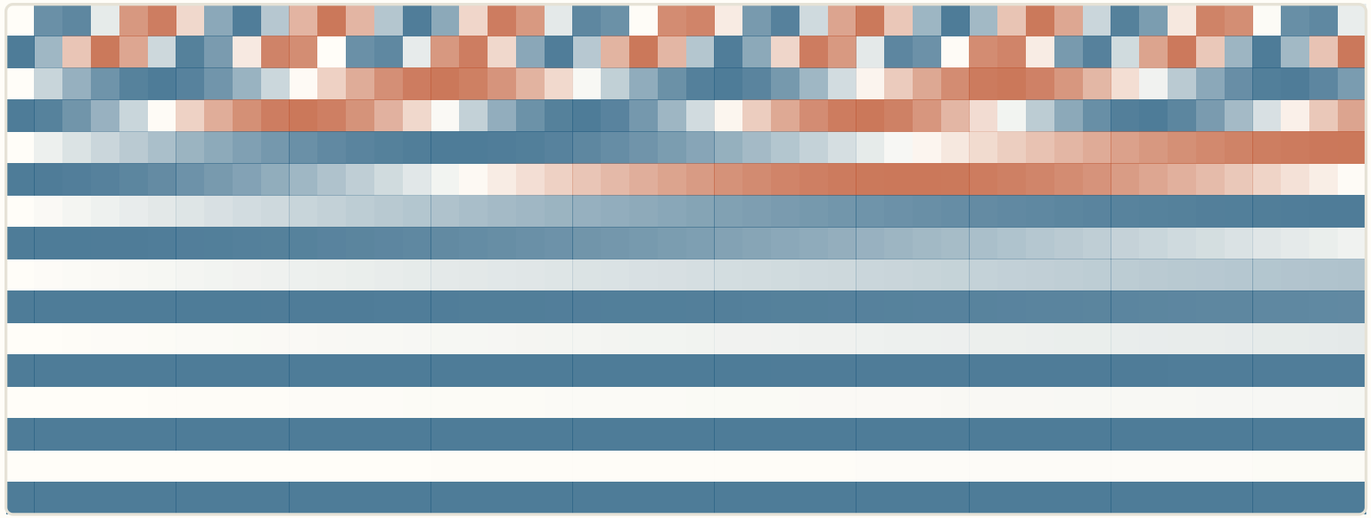
KẾT QUẢ

```
position 0: [0.0, 1.0, 0.0, 1.0]
equivariant with fixed slots: False
```

Tín hiệu vị trí cho model khả năng phân biệt thứ tự.

Sin/cos trên 48 vị trí

tần số cao



tần số thấp

xanh: dương

cam: âm

vị trí p = 0 … 47

Positional encoding: chiều đầu dao động nhanh, chiều sau chậm.

Relative position và bảng học được

Relative: score $\sqrt{}$.

Cùng khoảng cách, cùng vector ; hợp chuỗi thời gian.

Bảng học được: `nn.Embedding(max_len, d)`.

Chuỗi dài hơn `max_len` cần quy tắc trước: cắt hoặc chia cửa sổ.

HỎI NHANH

Bảng vị trí học được là `nn.Embedding(512, d)`. Tập test có một câu dài 600 token. Nếu không xử lý gì, chuyện gì xảy ra?

A Các vị trí từ 512 tới 599 tự nội suy từ những vị trí đã học

B Các vị trí từ 512 trở đi dùng chung vector của vị trí 511

C Tra vị trí từ 512 trở đi sẽ báo lỗi chỉ số

D Chạy bình thường, vì attention không giới hạn độ dài

Đường gradient có hoạt động?

KẾT QUẢ

```
finite loss: True
all parameter gradients finite: True
```

Mọi tham số nhận gradient hữu hạn: block sẵn sàng để train.

Attention weights là hệ số trộn, chưa là giải thích nhân quả.

HỎI NHANH

Trong model phân loại cảm xúc, một head ở lớp cuối dồn 0.9 trọng số vào từ “không”. Kết luận nào có cơ sở?

A Mới chỉ biết hệ số trộn, phải thử bỏ hoặc thay từ ấy mới kết luận được

B Từ “không” là nguyên nhân chính của dự đoán

C Head này chuyên xử lý phủ định, có thể bỏ các head khác

D Head này bị lỗi, vì trọng số tốt phải trải đều

Chi phí mọi cặp

Score và trộn: ; bộ nhớ score nếu lưu đủ.

gấp đôi → score gấp bốn.

Padding trả theo : bucketing, cắt, chia cửa sổ.

Linear attention: cho phép đổi thứ tự nhân, chi phí tuyến tính theo .

Số ô score theo



số ô score L × L của một head, một câu: L gấp đôi thì gấp bốn

Độ dài gấp 16 thì số ô của ma trận score gấp 256.

HỎI NHANH

Một batch 32 câu: 31 câu dài khoảng 100 token, một câu dài 2000 token. Cả batch được pad lên 2000. Cách rẻ nhất để giảm lãng phí là gì?

A Xếp những câu có độ dài gần nhau vào cùng batch

B Tăng số head để chia nhỏ ma trận score

C Bỏ `key_padding_mask` để khỏi phải tính phần PAD

D Giảm `embed_dim` để ma trận score nhỏ lại

Khi đi thi

- In mask 3×3, đọc quy ước của đúng hàm.
- Tính trước khi chọn `max_length`.
- Kiểm tổng hàng `weights` bằng 1.

05

Luyện đề

LUYỆN ĐỀ 1

Tự cài attention bằng `scores = Q @ K.transpose(-2, -1)` rồi softmax. Trọng số rất nhọn, khoảng 99% dồn vào một vị trí. Còn thiếu phép nào?

A Chia cho `self.head_dim`, tức số chiều được cộng

B Chia cho `math.sqrt(self.head_dim)`

C Chia cho `math.sqrt(self.num_heads)` để cân các head

D Thay bằng cosine của Q và K đã chuẩn hóa

ĐỀ THI NOAI 2026 Singapore · Chung kết · Câu 17

LUYỆN ĐỀ 2

Batch 8, 12 head, float32. Ở , score của một lớp chiếm khoảng 0.1 GB. Lên thì khoảng bao nhiêu?

A Không đổi, vì số tham số không phụ thuộc độ dài

B Khoảng 0.8 GB, tăng 8 lần như số token

C Khoảng 51 GB, tăng 512 lần

D Khoảng 6.4 GB, tăng 64 lần

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 247

LUYỆN ĐỀ 3

Tính , với , cỡ và cỡ .
Tính theo thứ tự nào thì chi phí tuyến tính theo ?

A Tính trước, rồi nhân với

B Tính trước, rồi nhân với

C Thứ tự nào cũng vậy, vì nhân ma trận có tính kết hợp

D Không có, vẫn phải tạo ma trận

PHỎNG THEO USA-NA-AIO 2026 · Vòng 2 · Ngày 1 · Part 1.1.3–1.1.4

Chốt ý

Attention: mỗi query lấy trung bình các value, trọng số từ độ khớp query–key chia $\sqrt{\ }$.

Padding mask bỏ phần điền, causal mask bỏ tương lai, mã vị trí giữ thứ tự; chi phí tăng theo bình phương độ dài.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook: chạy lại phép thử thay token tương lai và phép thử thêm PAD.